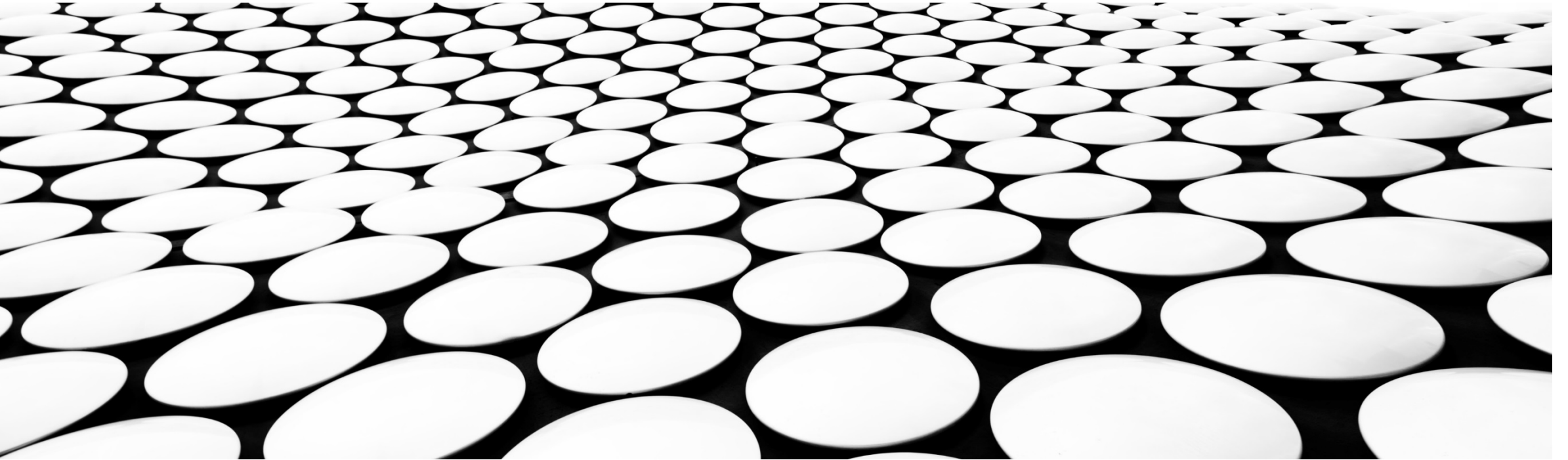

ARTIFICIAL INTELLIGENCE - STATE SPACE SEARCH



TIC TAC TOE

- **Nodes(N)**: all the different configuration of Xs and Os that the game can have.
- **Arcs (A)**: generated by legal moves by placing an X or an O in unused location.
- **Start state (S)**: an empty board.
- **Goal states (GD)**: a board state having three Xs in a row, column, or diagonal.
- The arcs are directed, then no cycles in the state space, directed acyclic graph (DAG).

TH 8-PUZZLE:

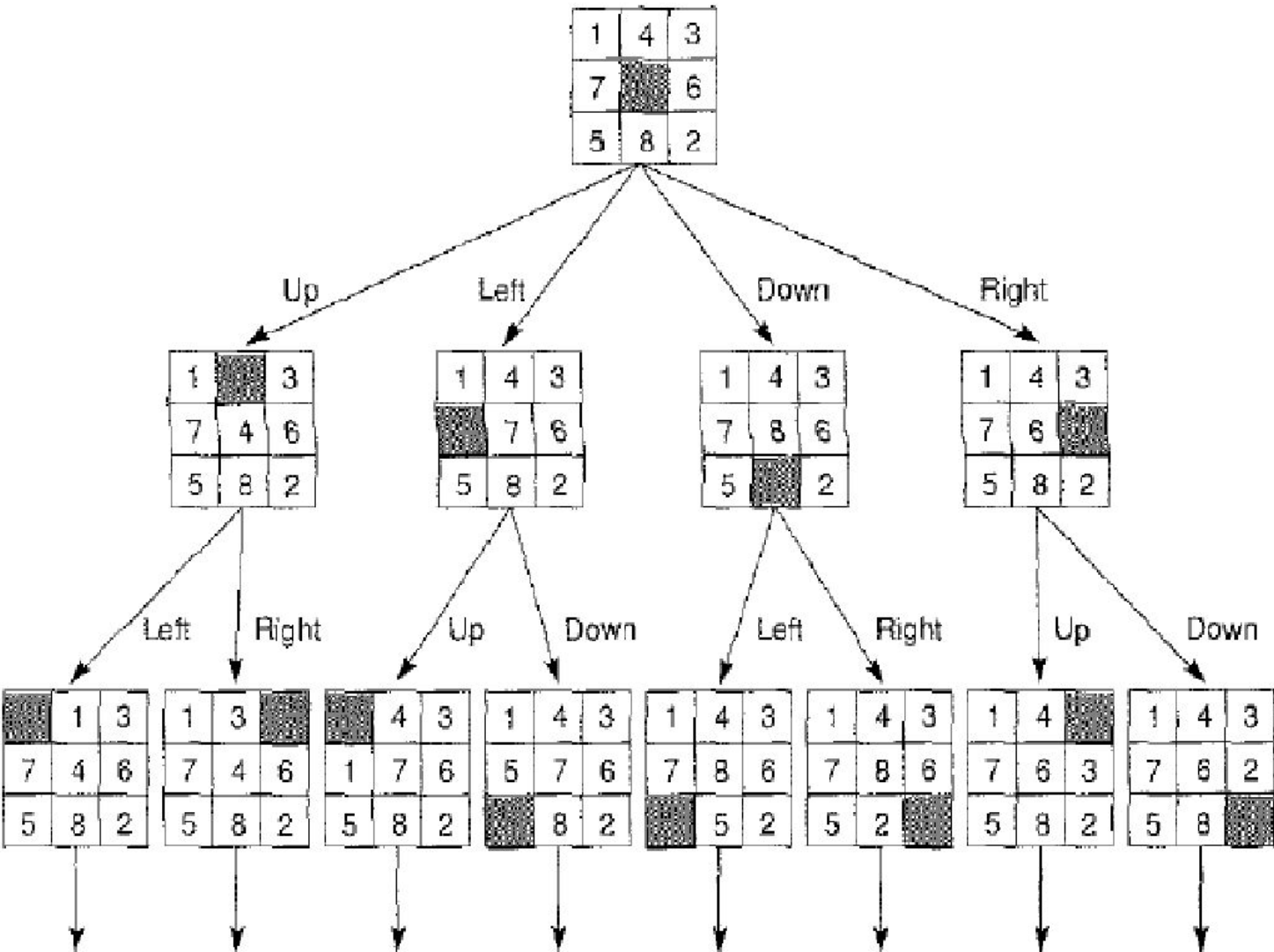


Figure 3.8 State space of the 8-puzzle generated by "move blank" operations.

THE TRAVELING SALESPERSON

- From city A, and return to the same city.
- Visit each city once

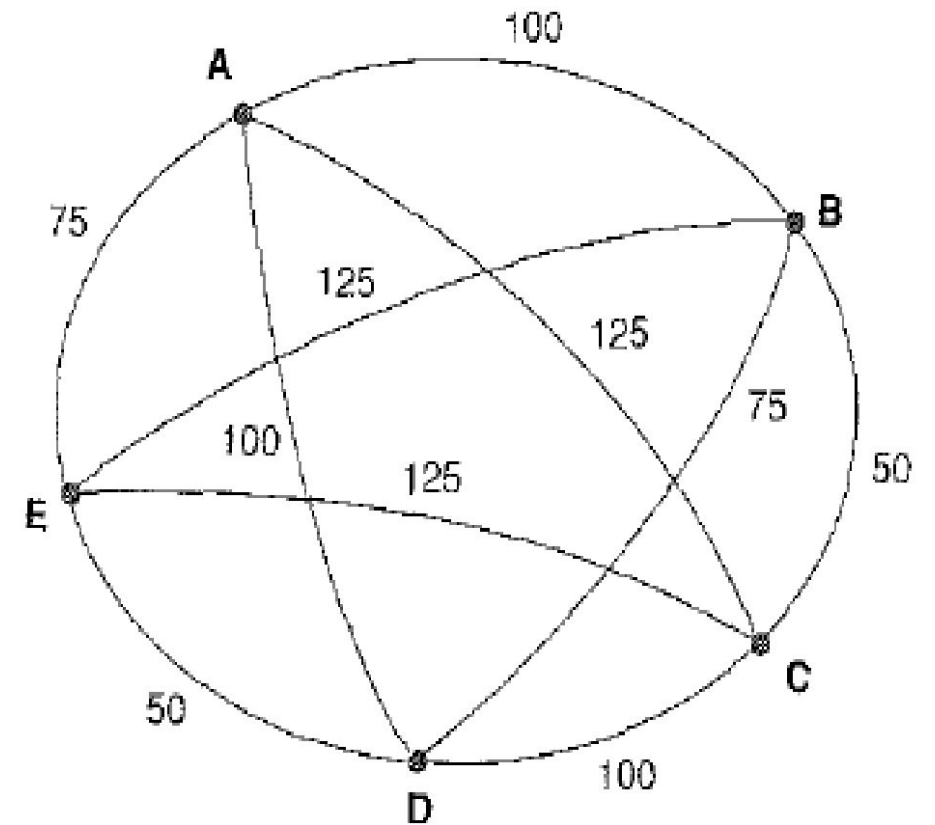


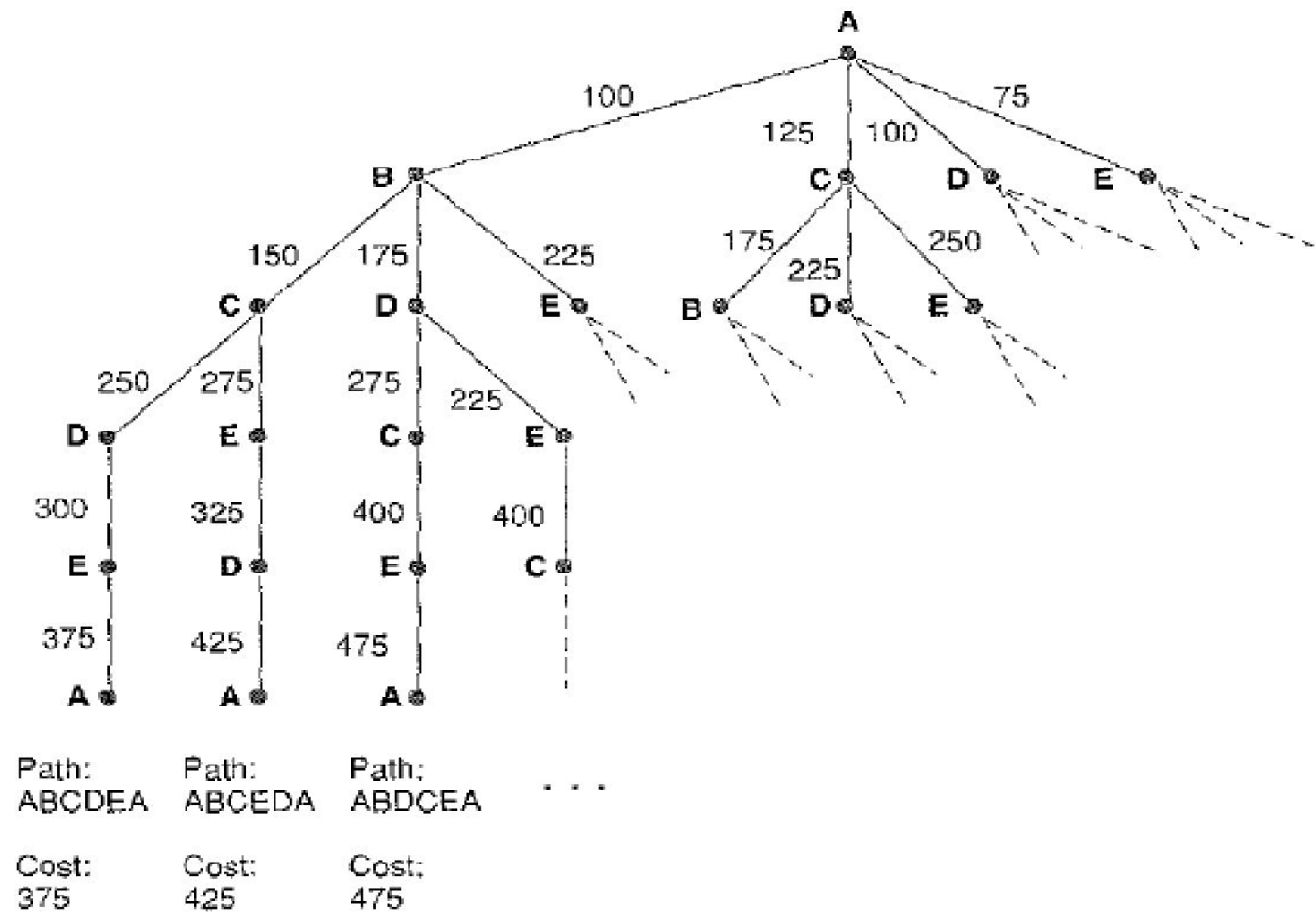
Figure 3.9 An instance of the traveling salesperson problem.

THE TSP

A salesperson has five cities to visit and then must return home.

- Nodes(N): represent 5 cities.
- Arcs(A): labeled with weight indicating the cost of traveling between connected cities.
- Start state(S): a home city.
- Goal states(GD): an entire path contains a complete circuit with minimum cost.
- Complexity: $(n-1)!$ Different cost-weighted paths can be generated.

CONT.



CONT.

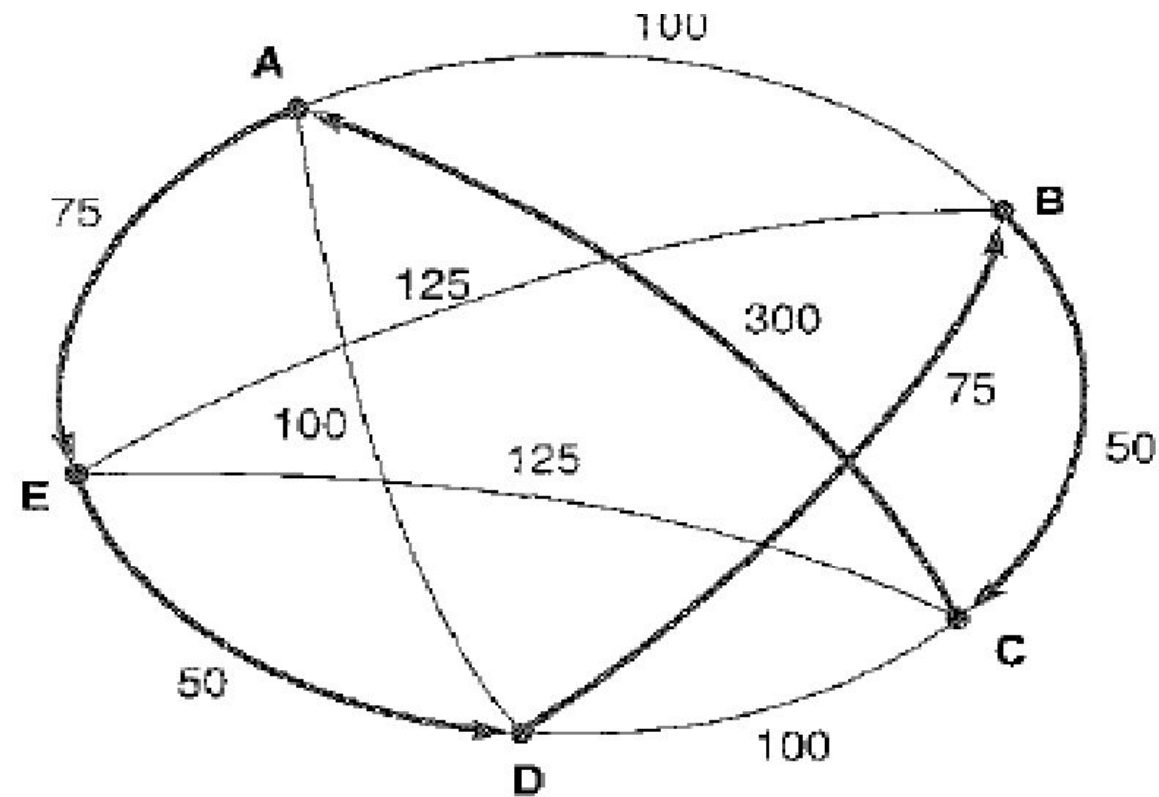


Figure 3.11 An instance of the traveling salesperson problem with the nearest neighbor path in bold. Note that this path (A, E, D, B, C, A), at a cost of 550, is not the shortest path. The comparatively high cost of arc (C, A) defeated the heuristic.

STRATEGIES FOR STATE SPACE SEARCH

■ Data-Driven

■ *Forward chaining*

- Begins with the given facts of the problem and a set of legal moves or rules for changing state.
- Search proceeds by applying rules to facts to produce new facts, which are in turn used by the rules to generate more new facts.
- This process continues until (we hope) it generates path that satisfies the goal condition.

■ Goal-Driven Search

- *goal-driven reasoning, or backward chaining*
- Take the goal that we want to solve. See what rules are legal moves could be used to generate this goal and determine what conditions must be true to use them.
- These conditions become the new goals, or subgoals, for the search.
- Search continues, working backward through successive subgoals until (we hope) it works back to the facts of the problem.

STRATEGIES FOR STATE SPACE SEARCH

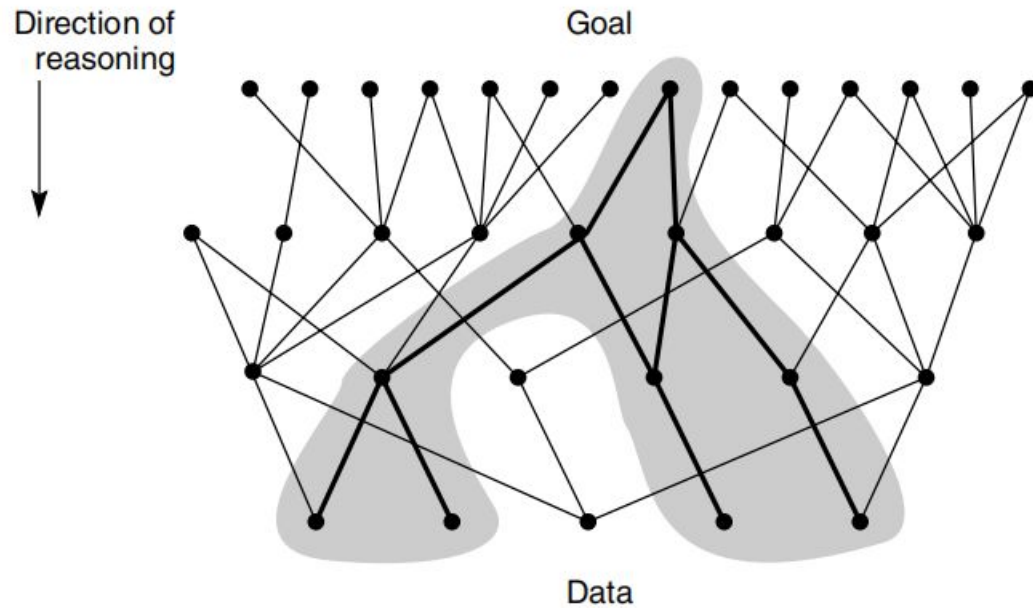


Figure 3.12 State space in which goal-directed search effectively prunes extraneous search paths.

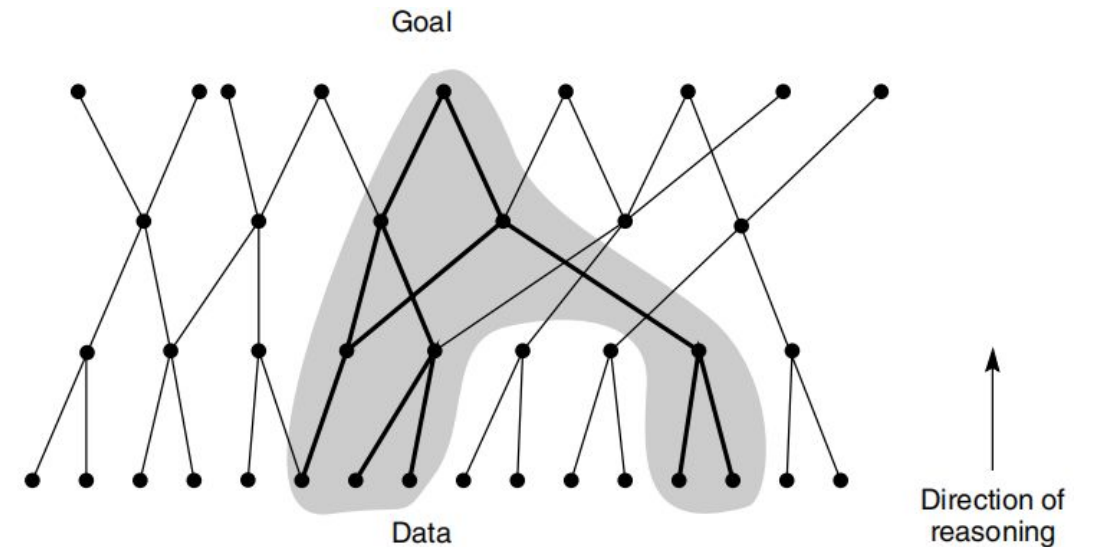
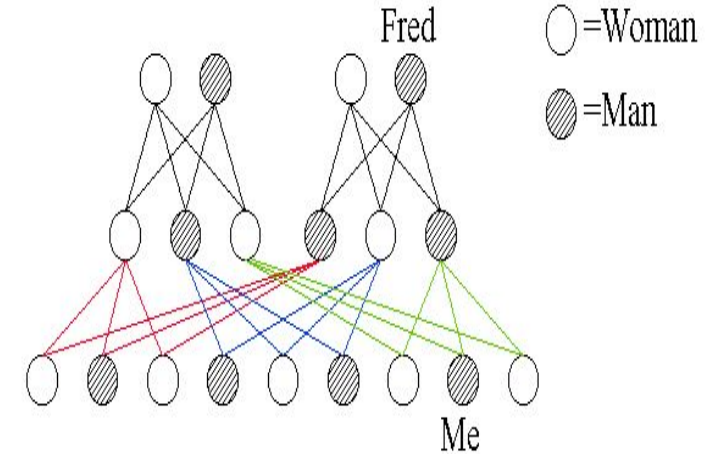


Figure 3.13 State space in which data-directed search prunes irrelevant data and their consequents and determines one of a number of possible goals.

DIRECTION OF SEARCH

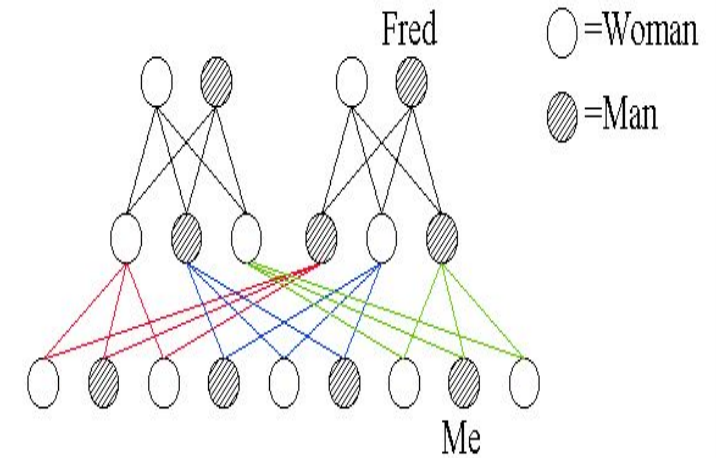


- Sometimes, the direction of the search will make a difference to the size of the search. It may be that there are less nodes to consider if you search back from the goal node up to the start node.
- A search algorithm attempting to find out if I am a descendant of Fred can take two different routes.
- It can start at Fred and work its way down, or it can start at me and work its way up. All I am interested is in finding a route between Fred and myself.
- So let's look at a top to bottom search, starting with Fred and attempting to find me.

DIRECTION OF SEARCH

- Going down the tree we see that there are three nodes to check at level one.
- From these three, there are a further nine nodes to check, so we may have to check up to 12 nodes, excluding Fred.
- If we go up the tree starting at Me, we see that one level up there are two nodes to examine, my parents.
- Going on from my parents, there are four more nodes - my grandparents of whom Fred is one.
- Going up the tree I only have to consider 6 nodes as opposed to 12.
- If John wants to find out if he is a descendant of Ethel, who is not two but 5 generations away the difference becomes even more noticeable.
- The top-down approach would have to consider up to 363 nodes whereas the bottom-up approach would only have to consider 62 at the most - a significant difference.

DIRECTION OF SEARCH



- Why is this?
- Well it's all in the branching factor,
 - for the top-down approach the branching factor is 3 - each couple has three children -so at level n there are 3^n nodes to consider.
 - The branching factor the other way is 2 as each person has only two parents, thus n levels up the tree from the start there are 2^n nodes to consider.
- So if you have a tree with one average branching factor one way and a smaller average the other way, and if you are free to solve the problem in either direction, it is probably advisable to try solving it going in the direction that gives the least branching factor.

GRAPH SEARCH: BACKTRACK SEARCH

- Backtracking search begins at the start state and pursues a path until it reaches either a goal or a "dead end." If it finds a goal, it quits and returns the solution path. If it reaches a dead end, it "backtracks" to the most recent node on the path having unexamined siblings and continues down one of these branches, as described in the following recursive rule:
 - *If the present state S does not meet the requirements of the goal description, then generate its first descendant S_{child1} and apply the backtrack procedure recursively to this node. If backtrack does not find a goal node in the subgraph rooted at S_{child1} , repeat the procedure for its sibling, S_{child2} . This continues until either some descendant of a child is a goal node or all the children have been searched. If none of the children of S leads to a goal, then backtrack "fails back" to the parent of S , where it is applied to the siblings of S , and so on,*

IMPLEMENTING GRAPH SEARCH: BACKTRACK ALGORITHM

```
function backtrack;
begin
  SL := [Start]; NSL := [Start]; DE := [ ]; CS := Start;           % initialize:
  while NSL ≠ [ ] do                                             % while there are states to be tried
  begin
    if CS = goal (or meets goal description)
    then return SL;                                             % on success, return list of states in path.
    if CS has no children (excluding nodes already on DE, SL, and NSL)
    then begin
      while SL is not empty and CS = the first element of SL do
      begin
        add CS to DE;                                           % record state as dead end
        remove first element from SL;                           % backtrack
        remove first element from NSL;
        CS := first element of NSL;
      end
      add CS to SL;
    end
    else begin
      place children of CS (except nodes already on DE, SL, or NSL) on NSL;
      CS := first element of NSL;
      add CS to SL
    end
  end
end;
return FAIL;
end.
```

CONT.

- **SL**, for state list, lists the states in the current path being tried. If a goal is found, SL contains the ordered list of states on the solution path.
- **NSL**, for new state list, contains nodes awaiting evaluation, i.e., nodes whose descendants have not yet been generated and searched.
- **DE**, for dead ends, lists states whose descendants have failed to contain a goal node. If these states are encountered again, they will be detected as elements of DE and eliminated from consideration immediately.
- **CS** for current state. CS is always equal to the state most recently added to SL and represents the "frontier" of the solution path currently being explored.

EXAMPLE

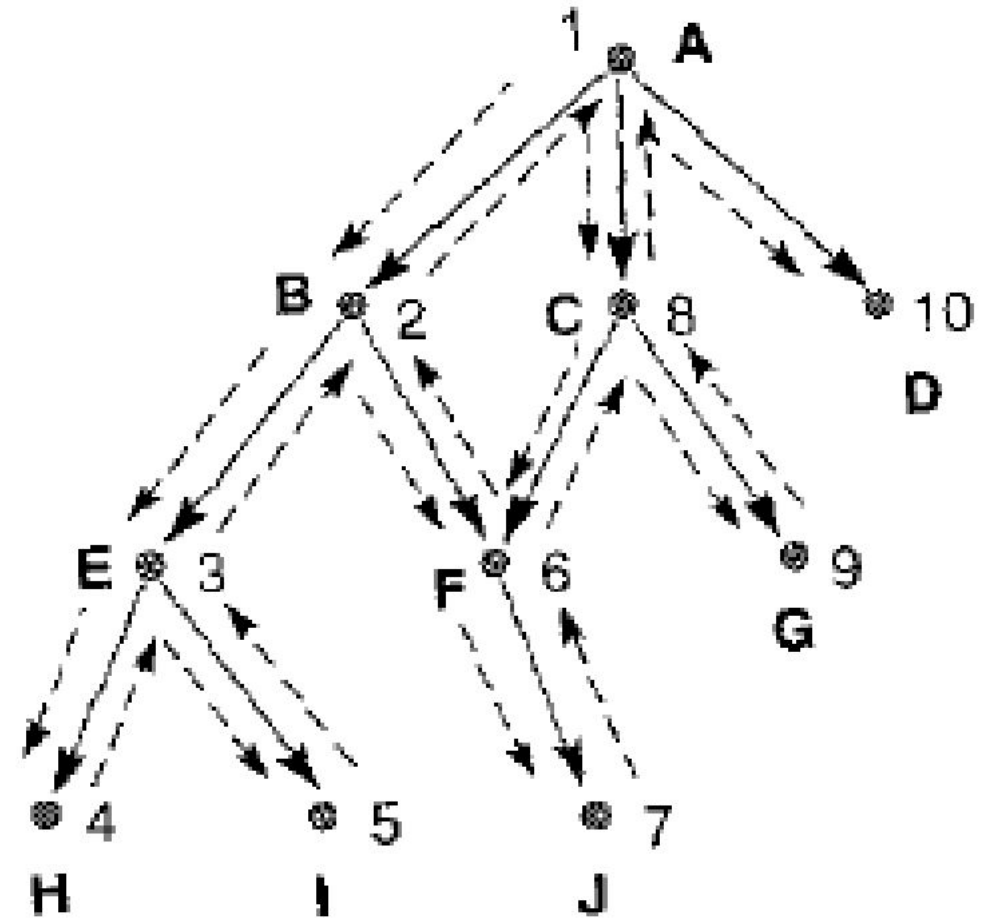


Figure 3.14 Backtracking search of a hypothetical state space space.

Initialize: SL = {A}; NSL = {A}; DE = []; CS = A;

TRACE

AFTER ITERATION	CS	SL	NSL	DE
0	A	[A]	[A]	[]
1	B	[B A]	[B C D A]	[]
2	E	[E B A]	[E F B C D A]	[]
3	H	[H E B A]	[H I E F B C D A]	[]
4	I	[I E B A]	[I E F B C D A]	[H]
5	F	[F B A]	[F B C D A]	[E I H]
6	J	[J F B A]	[J F B C D A]	[E I H]
7	C	[C A]	[C D A]	[B F J E I H]
8	G	[G C A]	[G C D A]	[B F J E I H]

CONT.

- backtrack is an algorithm for searching state space graphs. The graph search algorithms in the remainder of the text, including depth-first breadth-first, and best-first search, exploit the ideas used in backtrack. including:
 - The use of a list of unprocessed states (NSL) to allow the algorithm to return (backtrack) to any of these states.
 - A list of "had" states (DE) to prevent the algorithm from retrying useless paths.
 - A list of nodes (SL) on the current solution path that is returned if a goal is found.
 - Explicit checks for membership of new states in these lists to prevent looping.

DEPTH-FIRST AND BREADTH-FIRST SEARCH

- In addition to specifying a search direction (data-driven or goal-driven), a search algorithm must determine the order in which states are examined in the tree or the graph

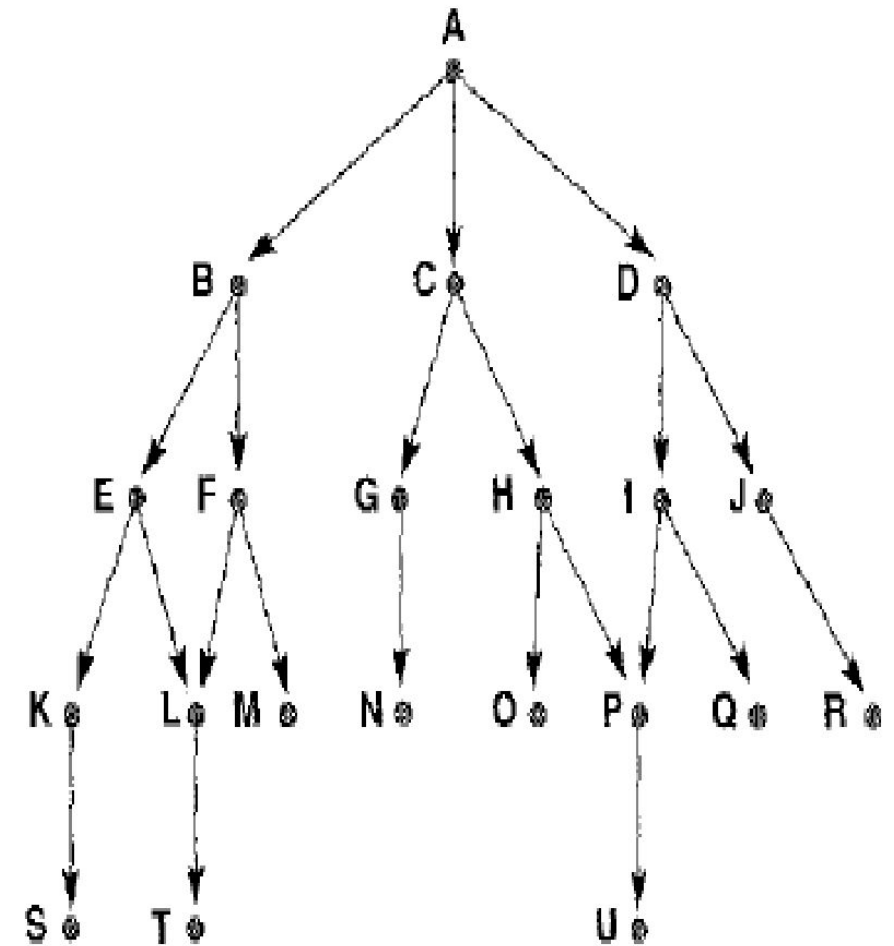


Figure 3.15 Graph for breadth- and depth-first search examples.

BFS RUN

1. open = [A]; closed = []
2. open = [B,C,D]; closed = [A]
3. open = [C,D,E,F]; closed = [B,A]
4. open = [D,E,F,G,H]; closed = [C,B,A]
5. open = [E,F,G,H,I,J]; closed = [D,C,B,A]
6. open = [F,G,H,I,J,K,L]; closed = [E,D,C,B,A]
7. open = [G,H,I,J,K,L,M] (as L is already on open); closed = [F,E,D,C,B,A]
8. open = [H,I,J,K,L,M,N]; closed = [G,F,E,D,C,B,A]
9. and so on until either U is found or open = [].

open = [(D,A), (E,B), (F,B), (G,C), (H,C)]; closed = [(C,A), (B,A), (A,nil)]

CONT.

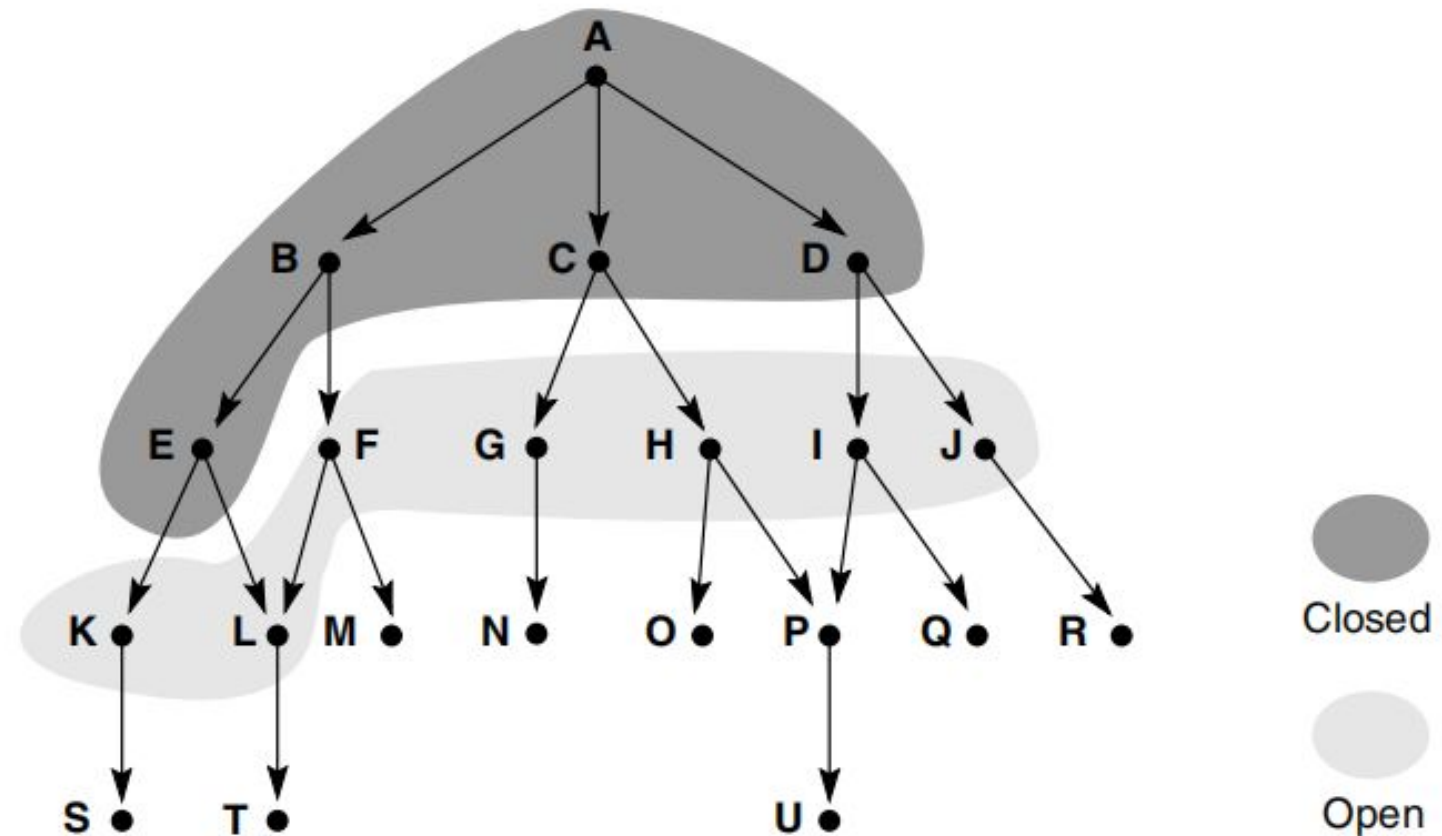


Figure 3.16 Graph of Figure 3.15 at iteration 6 of breadth-first search. States on open and closed are highlighted.

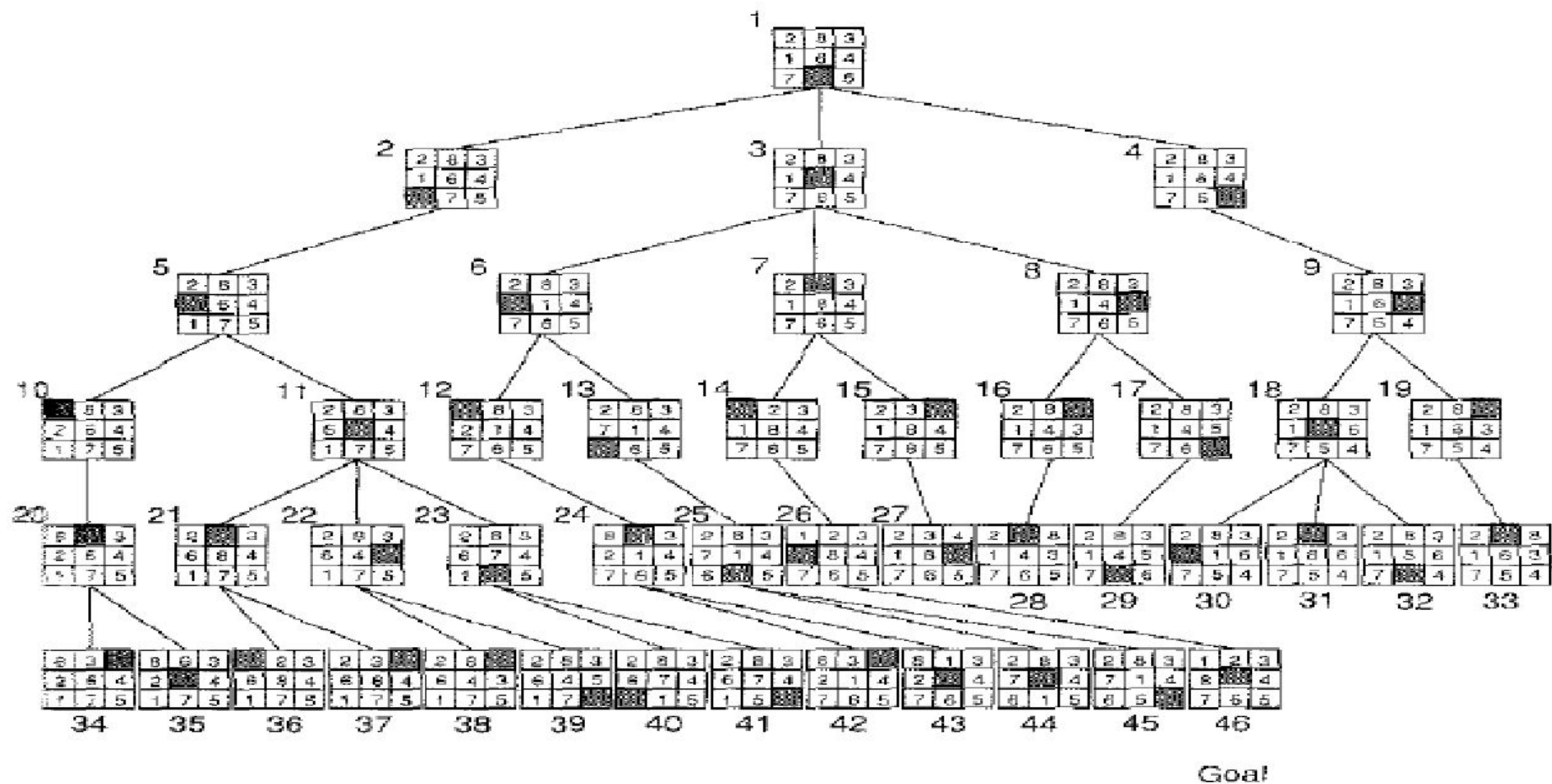


Figure 3.17 Breadth-first search of the 8-puzzle, showing order in which states were removed from open.

Depth	Nodes	Time	Memory
2	110	.11 milliseconds	107 kilobytes
4	11,110	11 milliseconds	10.6 megabytes
6	10^6	1.1 seconds	1 gigabyte
8	10^8	2 minutes	103 gigabytes
10	10^{10}	3 hours	10 terabytes
12	10^{12}	13 days	1 petabyte
14	10^{14}	3.5 years	99 petabytes
16	10^{16}	350 years	10 exabytes

Figure 3.13 Time and memory requirements for breadth-first search. The numbers shown assume branching factor $b = 10$; 1 million nodes/second; 1000 bytes/node.

- Exponential-complexity search problems cannot be solved by uninformed methods for any but the smallest instances.

```
function depth_first_search;
```

DFS

begin

```
open := [Start];
```

```
% initialize
```

```
closed := [ ];
```

```
while open  $\pi$  { } do
```

% states remain

begin

remove leftmost state from open, call it X;

```

if X is a goal then return SUCCESS

```

% goal found

```
else begin
```

generate children of X;

```
put X on closed;
```

discard children of X if already on open or closed:

```
% loop check
```

put remaining children on left end of open

```
% stack
```

end

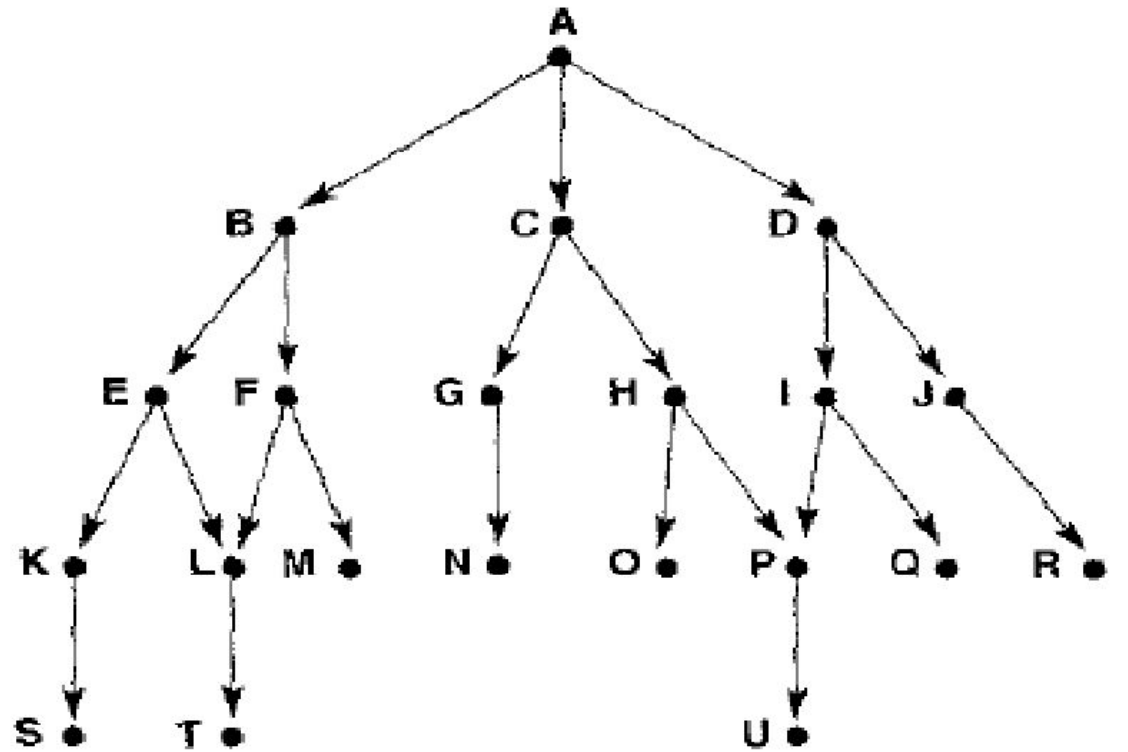
end;

```
return FAIL
```

% no states left

end.

CONT.



Closed

Open

Figure 3.18 Graph of Figure 3.15 at iteration 6 of depth-first search. States on open and closed are highlighted.

-
1. open = {A}; closed = []
 2. open = {B,C,D}; closed = {A}
 3. open = {E,F,C,D}; closed = {B,A}
 4. open = {K,L,F,C,D}; closed = {E,B,A}
 5. open = {S,L,F,C,D}; closed = {K,E,B,A}
 6. open = {L,F,C,D}; closed = {S,K,E,B,A}
 7. open = {T,F,C,D}; closed = {L,S,K,E,B,A}

 8. open = {F,C,D}; closed = {T,L,S,K,E,B,A}
 9. open = {M,C,D}, (as L is already on closed); closed = {F,T,L,S,K,E,B,A}
 10. open = {C,D}; closed = {M,F,T,L,S,K,E,B,A}
 11. open = {G,H,D}; closed = {C,M,F,T,L,S,K,E,B,A}

DFS

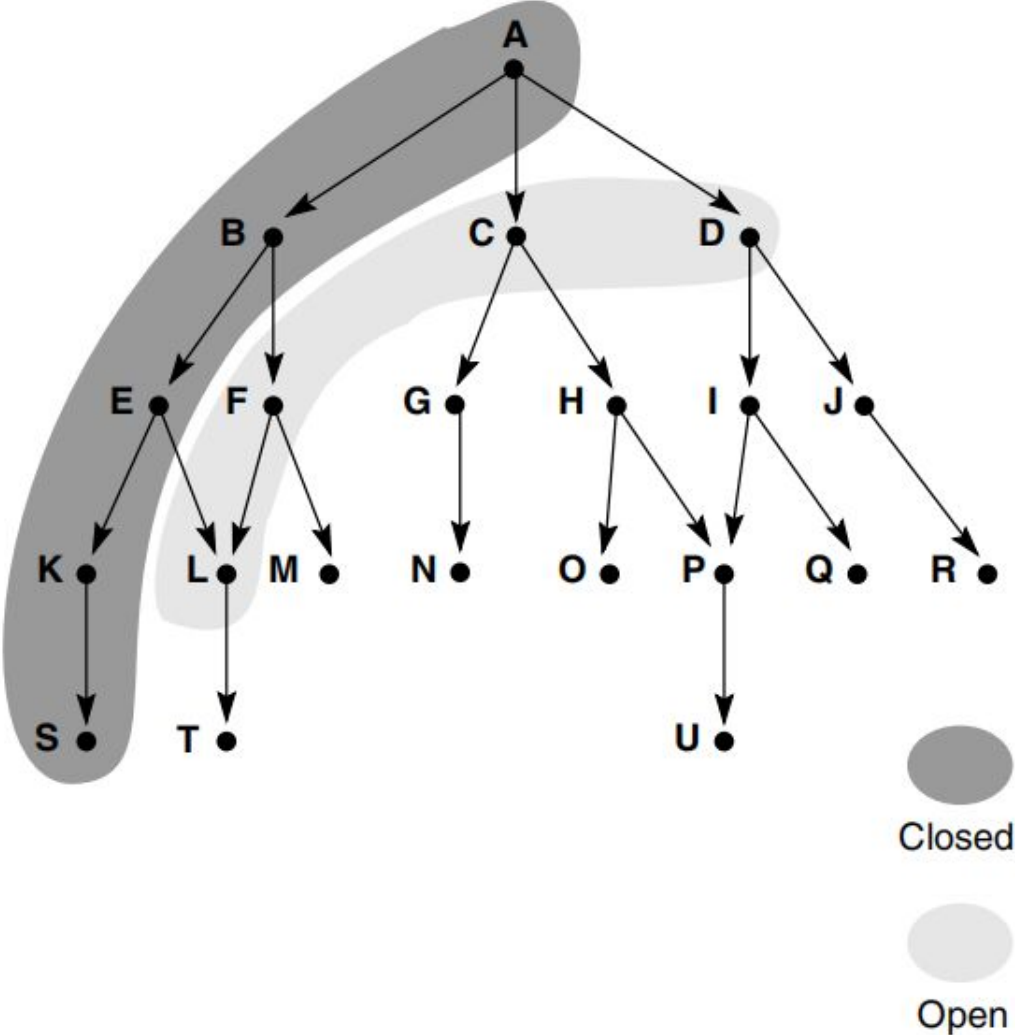


Figure 3.18 Graph of Figure 3.15 at iteration 6 of depth-first search. States on open and closed are highlighted.

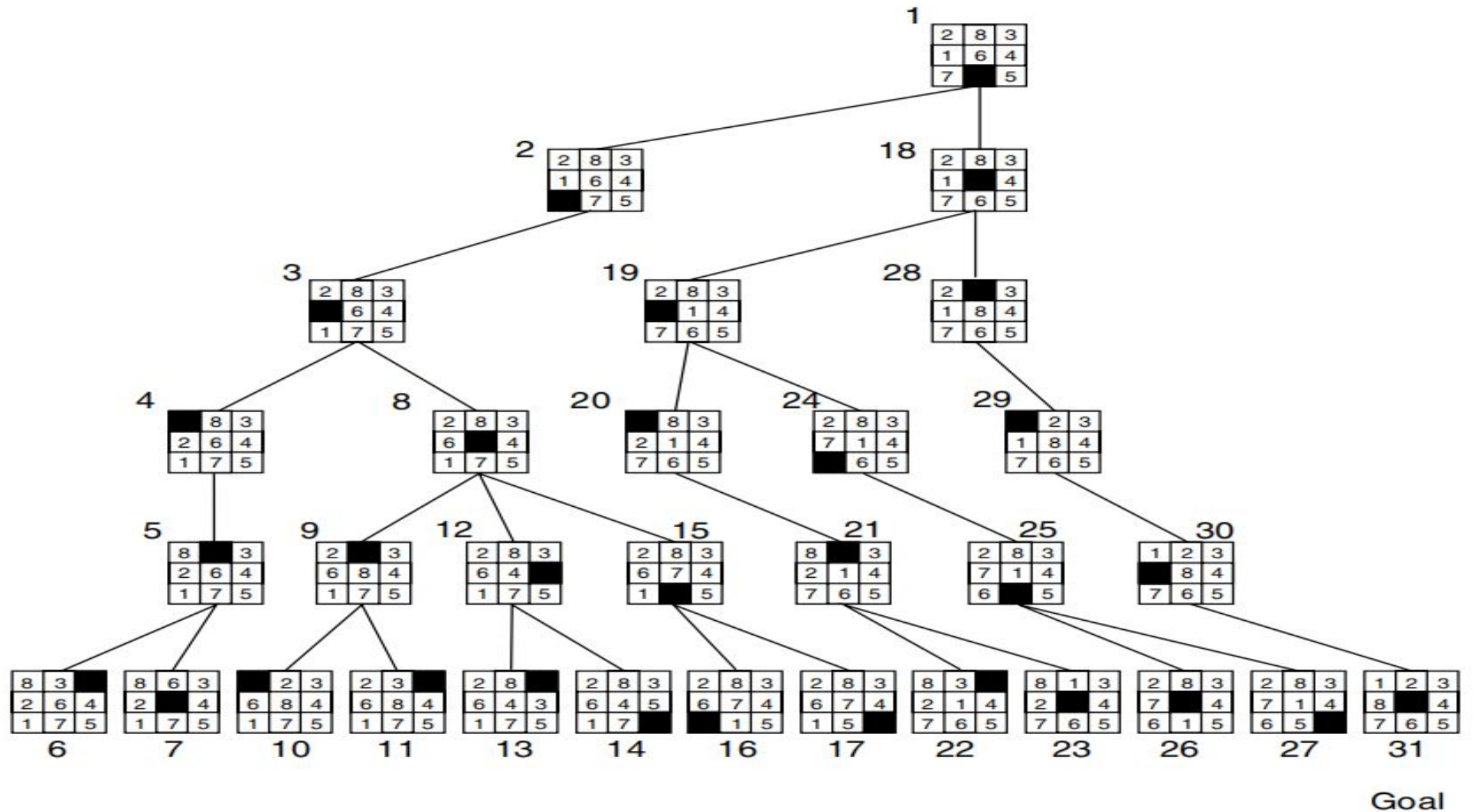


Figure 3.19 Depth-first search of the 8-puzzle with a depth bound of 5.



DFS

- If DFS goes down an infinite branch it will not terminate if it does not find a goal state.
- If it does find a solution, there may be a better solution at a lower level in the tree. Therefore, depth first search is neither complete nor optimal.
- • DFS may never terminate as it could follow a path that has no solution (e.g., Chess: repeated moves result in similar configuration)

BFS VS. DFS

- As with choosing between data- and goal-driven search for evaluating a graph, the choice of depth-first or breadth-first search depends on the specific problem being solved.
- Significant features include the importance of
 - finding the shortest path to a goal,
 - the branching factor of the space,
 - the available compute time and space resources,
 - the average length of paths to a goal node,
 - and whether we want all solutions or only the first solution.
- In making these decisions, there are advantages and disadvantages for each approach.

BFS VS. DFS

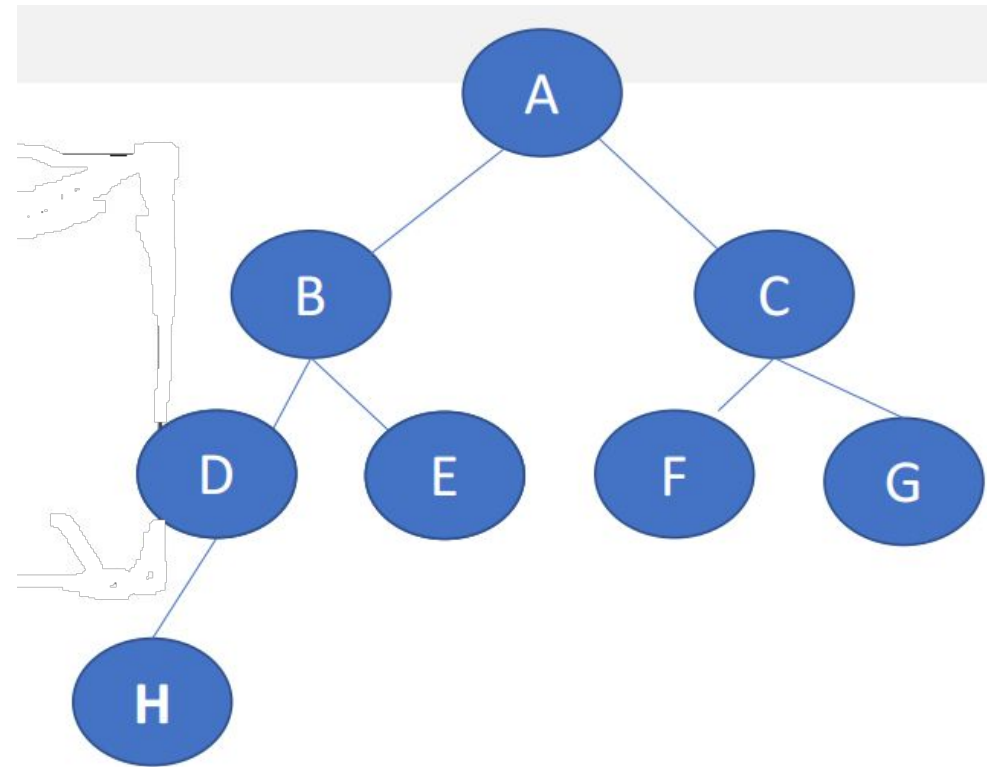
- BFS always examines all the nodes at level n before proceeding to level $n + 1$,
- In a problem where it is known that a simple solution exists, this solution will be found
- breadth-first search always finds the shortest path to a goal node
- space utilization of BFS, measured in terms of the number of states on open, is an exponential function of the length of the path at any time
- B^n states on level n , BFS would place all of these on open when it begins examining level n
- Depth-first search gets quickly into a deep search space.
- If it is known that the solution path will be long, depth-first search will not waste time searching a large number of “shallow” states in the graph
- On the other hand, depth-first search can get “lost” deep in a graph, missing shorter paths to a goal or even becoming stuck in an infinitely long path that does not lead to a goal.
- DFS is much more efficient for search spaces with many branches because it does not have to keep all the nodes at a given level on the open list.
- The space usage of DFS is a linear function of the length of the path
- If a graph has an average of B children per state, this requires a total space usage of $B \times n$ states to go n levels deep into the space.

DEPTH-FIRST SEARCH WITH ITERATIVE DEEPENING

- This form of search is an excellent compromise between depth-first and breadth-first searches.
- The search sets a depth limit/bound and then does a depth-first search down to this limit - if a solution is found it exits with success.
- If a solution is not found then the limit is increased and the search run again but down to this new depth.
- Obviously, a large number of nodes are revisited a number of times, but the overall loss due to this becomes insignificant for a search through a large tree.
- The number of nodes grows exponentially as you go down the tree so almost all the time is spent at the deepest levels which are only visited once unless the search repeats to a deeper level, at which the time spent in these levels will become insignificant.
- The memory consumption is the same as that for a depth-first search, but the search is always guaranteed to find a solution, as in breadth-first search.

ITERATIVE DEEPENING SEARCH

guaranteed to find a shortest path to a goal.
Because it does only depth-first search at each iteration,
the space usage at any level n is $B \times n$, where
 B is the average number of children of
a node.



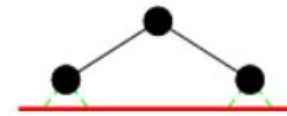
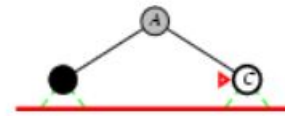
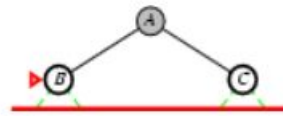
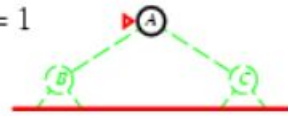
ITERATIVE DEEPENING SEARCH D=0

Limit = 0



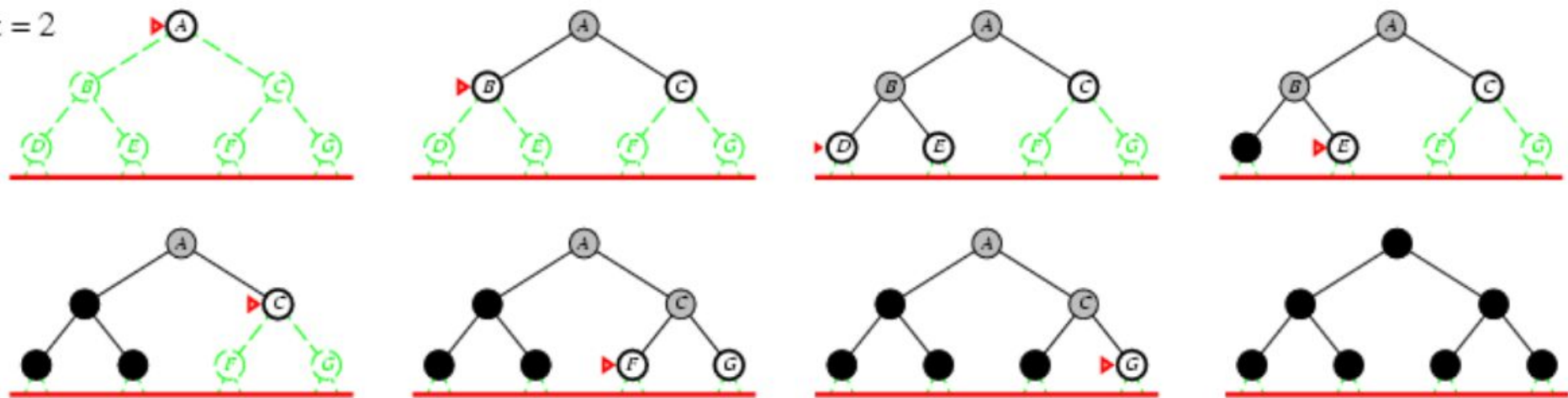
ITERATIVE DEEPENING SEARCH D=1

Limit = 1



ITERATIVE DEEPENING SEARCH D=2

Limit = 2



ITERATIVE DEEPENING SEARCH D=3

Limit = 3

